



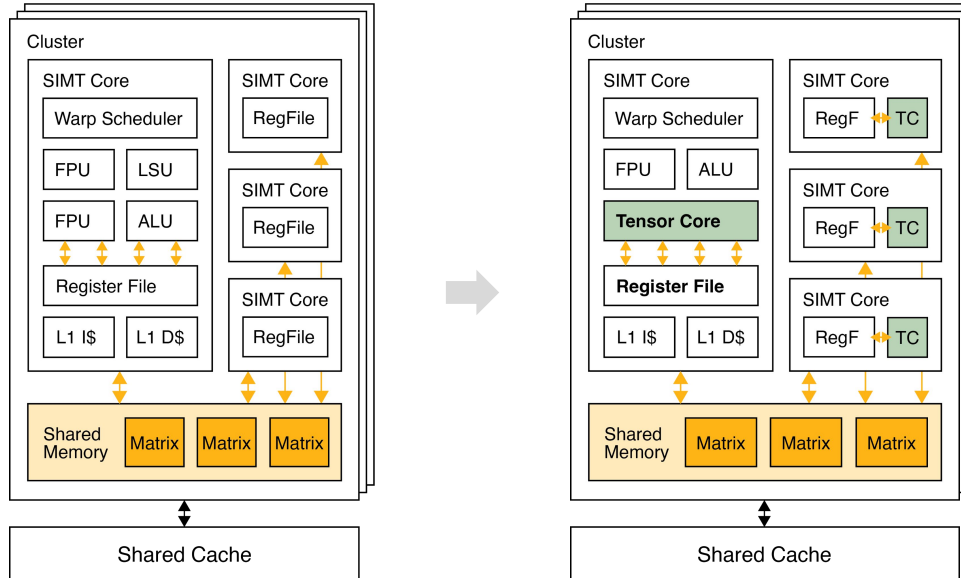
Radiance: Evolving GPUs for Asynchronous Decoupled Specialized Units

Hansung Kim, Richard Yan, Shashank Anand, Christopher Fletcher, Sophia Shao
SLICE Summer Retreat | May 21, 2025



Evolution of Modern GPUs

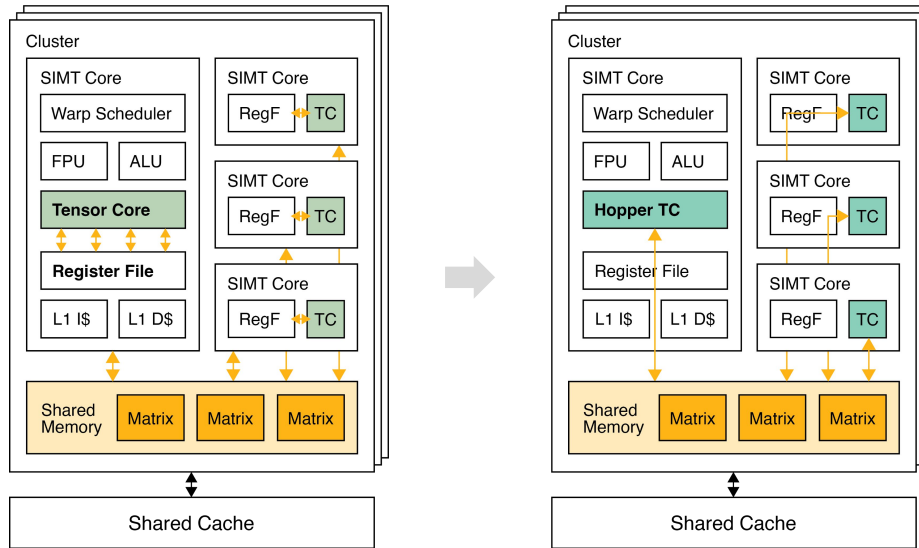
GPGPUs started as general-purpose architecture of SIMT cores running fine-grained insns
With emerging compute-intensive AI applications, modern GPUs integrated **domain-specialized units** for higher **compute density** and **efficiency**



Examples:
NVIDIA Tensor Cores,
AMD Matrix Cores,
Intel XMX engines

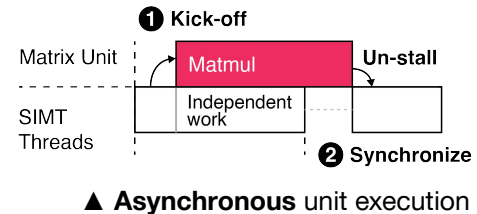
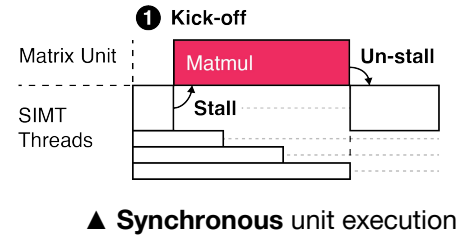
Evolution of Modern GPUs

Growing compute demand introduced larger specialized units with longer latencies not fully hidden by SIMT multithreading → **Asynchronous** specialized units



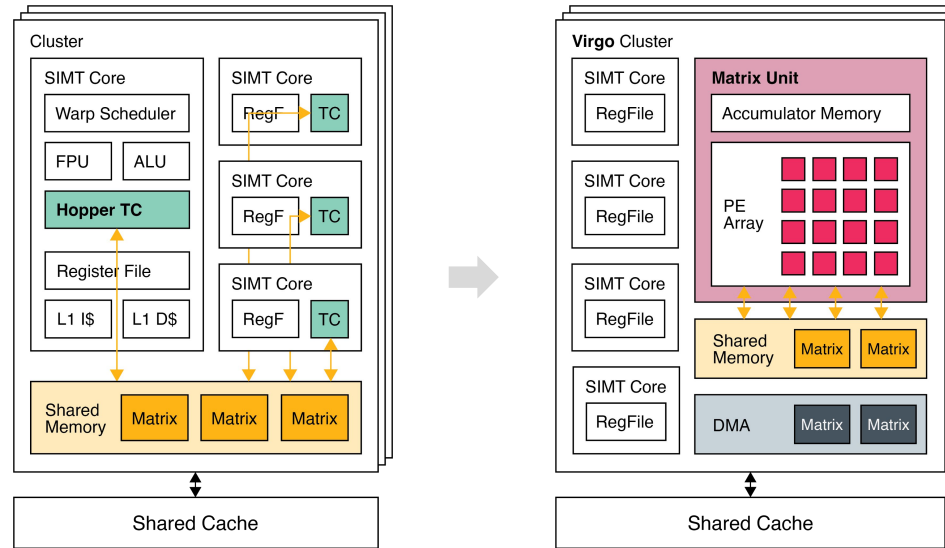
▲ **Synchronous** Units
e.g. Volta Tensor Cores

▲ **Asynchronous** Units
e.g. Hopper Tensor Cores



Evolution of Modern GPUs

Asynchronous design enabled **decoupled microarchitecture** of specialized units, overcoming scalability constraints of coupled designs

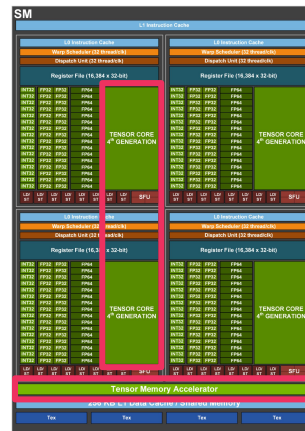
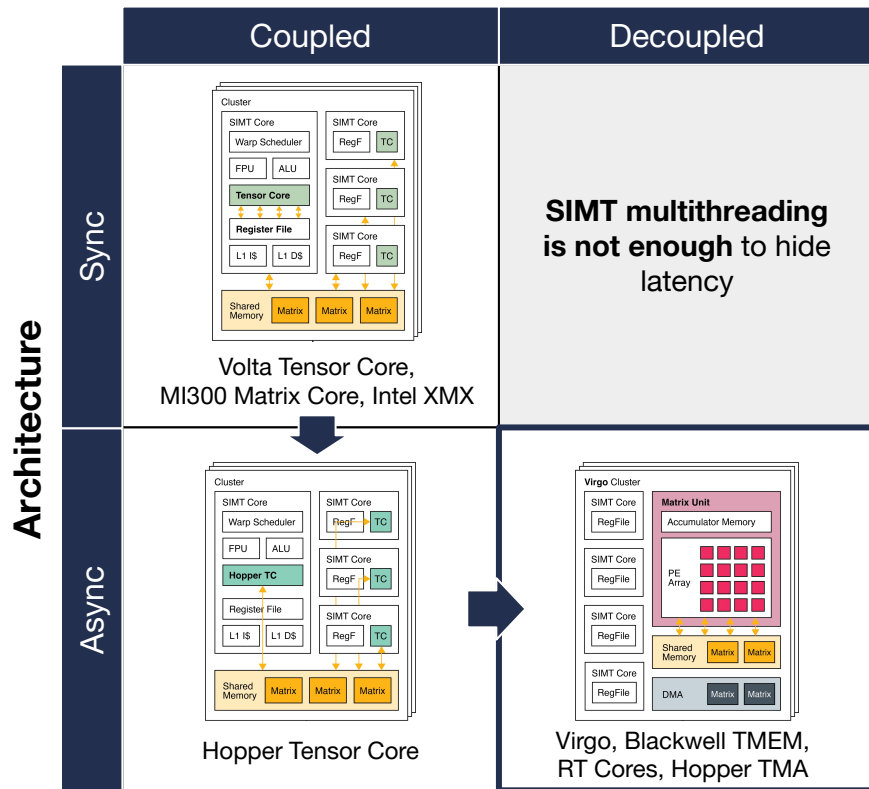


▲ **Asynchronous Coupled Units**
e.g. Hopper Tensor Cores

▲ **Asynchronous Decoupled Units**
e.g. Virgo, NVIDIA RT Cores,
Hopper TMA, Blackwell TMEM

Taxonomy of Specialized Units in GPUs

Microarchitecture



▲ NVIDIA Hopper

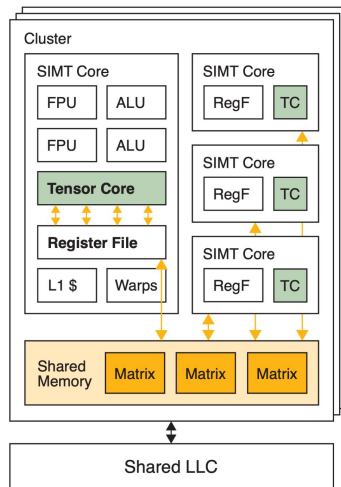


▲ Intel Xe-HPG

Future GPUs will feature diverse **asynchronous decoupled** units to meet scalability/efficiency demand

Virgo: Asynchronous Decoupled Matrix Units

Previous: **Coupled integration**

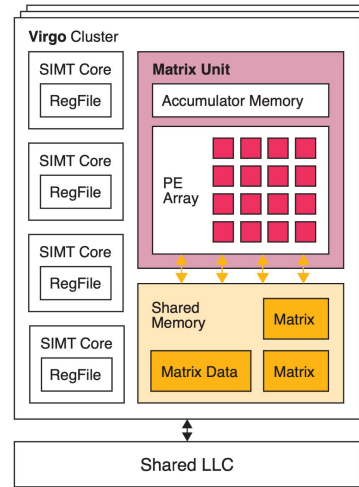


Register file caused

- ① limited tile size,
- ② low warp occupancy,
- ③ limited data re-use,
- ④ high insn overhead



Virgo: **Cluster-level integration**



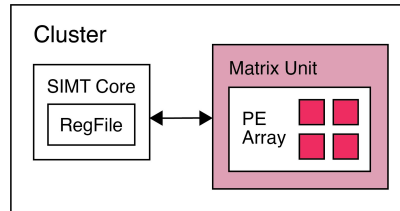
Virgo identified **scalability** and **efficiency** benefits of **asynchronous, decoupled** designs, but did not address how to efficiently **interface and orchestrate** decoupled units



Orchestrating Decoupled Units

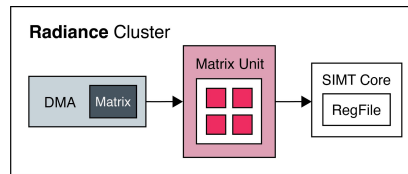
How to **control** and **synchronize** units without SIMT intervention?

1

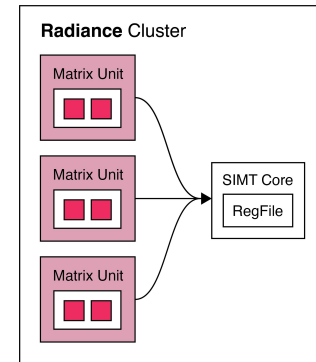


How to compose units into flexible **dataflow graphs** and **dispatch** ops in dependency order?

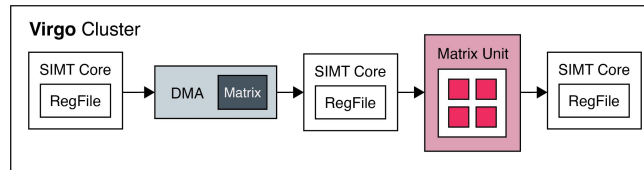
2



▲ GEMM with pipelined DMA



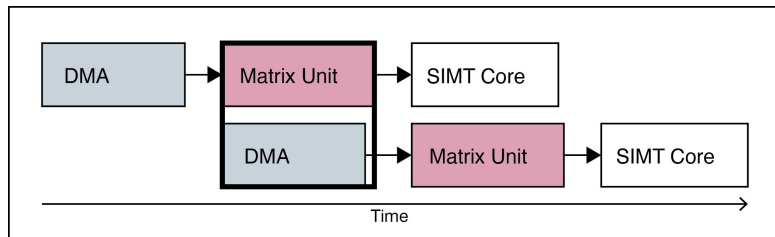
▲ GEMM with split-K



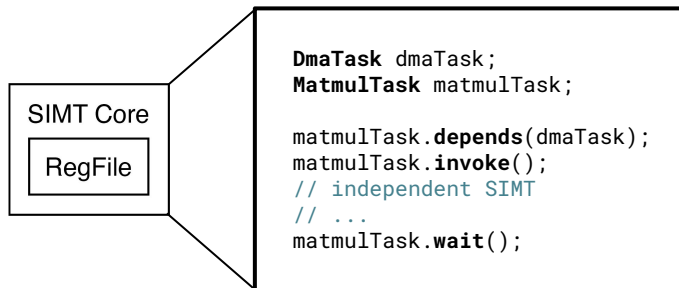


Orchestrating Decoupled Units

How to **schedule** units to maximize parallelism and hardware utilization? **3**



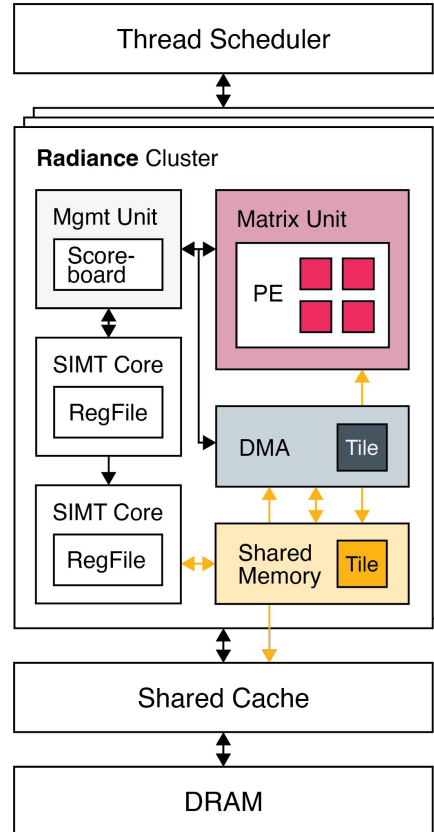
How to expose operations to the **programming model** without breaking SIMT? **4**



Goal: Develop a scalable mechanism for **orchestrating asynchronous decoupled units** in GPUs with high efficiency, while maintaining SIMT abstraction



Radiance GPU Architecture



Applies Virgo's **cluster-level decoupled** integration

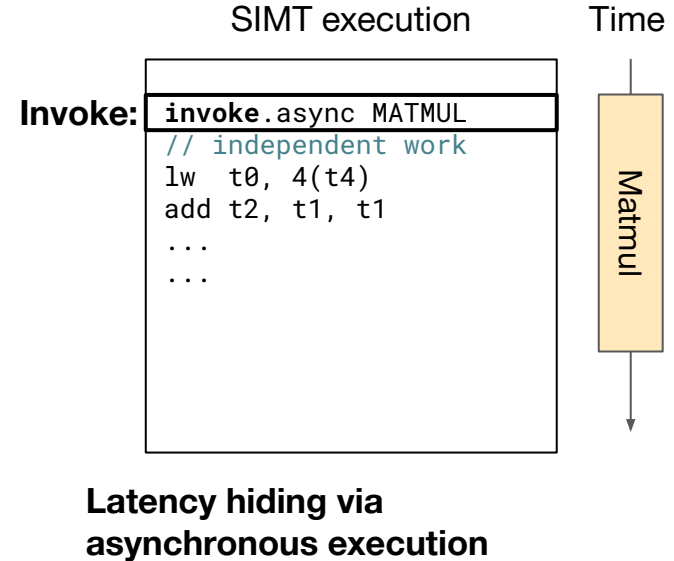
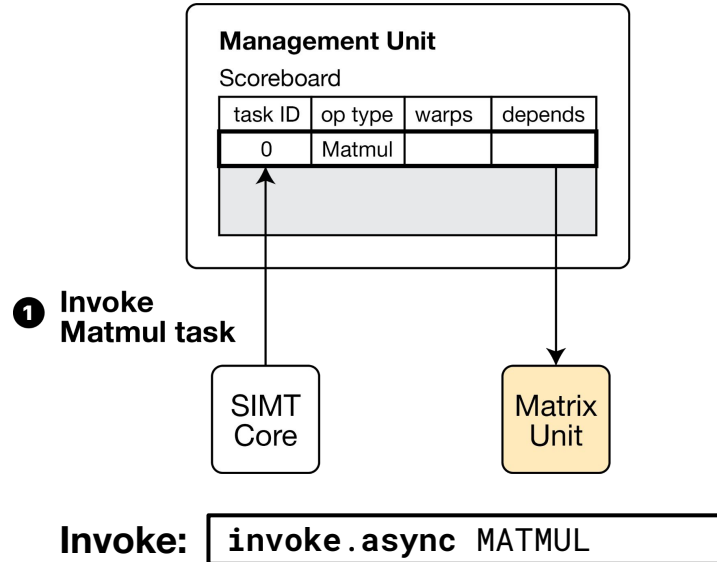
Dedicated **management unit** orchestrates **control** of SIMT + specialized units

SIMT and specialized units communicate through **shared memory**



Mgmt Unit | Control and Synchronization

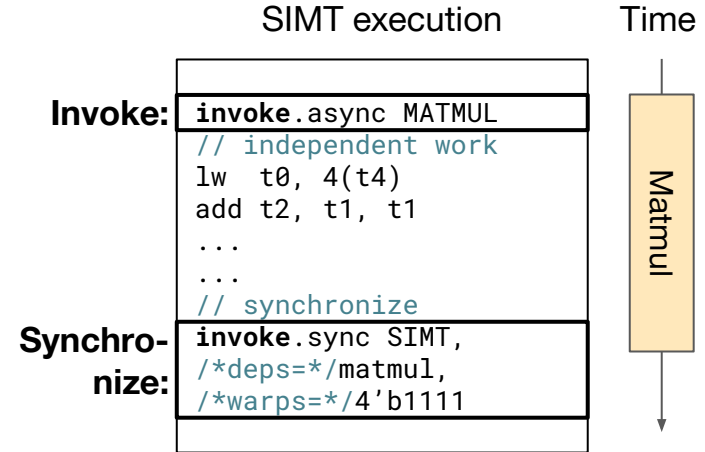
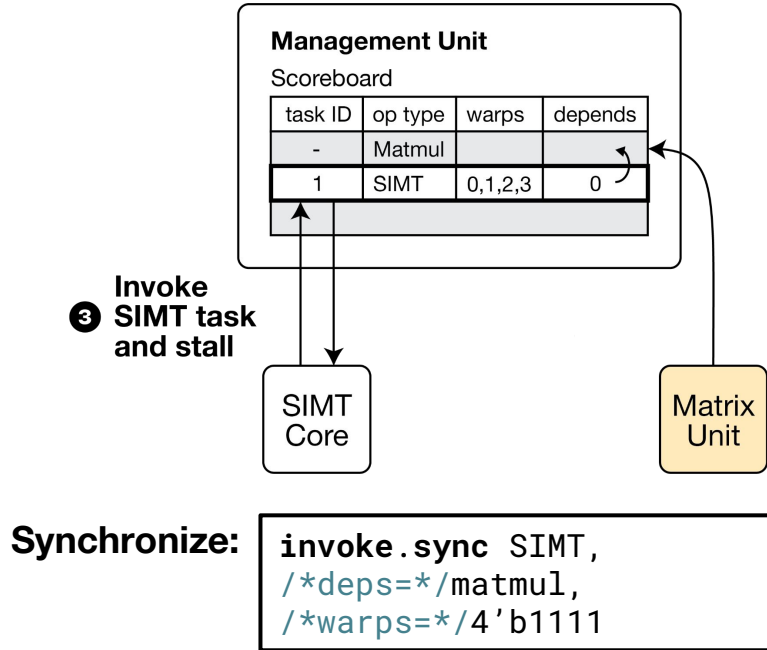
Management unit **① offloads control** and **synchronization** of units away from SIMT





Mgmt Unit | Control and Synchronization

Management unit **① offloads control** and **synchronization** of units away from SIMT



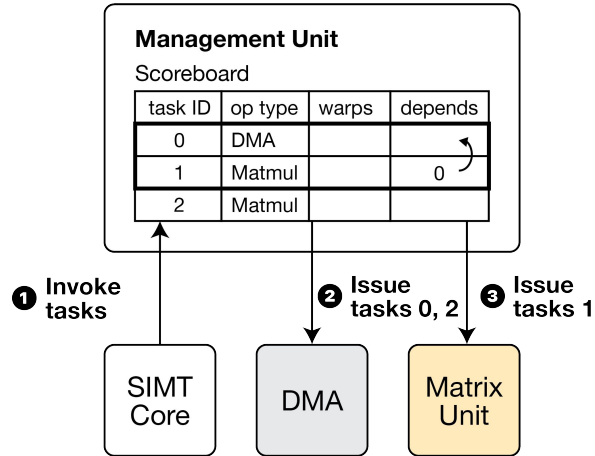
Minimal *fire-and-forget* SIMT interface

Matmul utilization not bottlenecked by SIMT insn throughput;
SIMT freed-up for useful vector ops



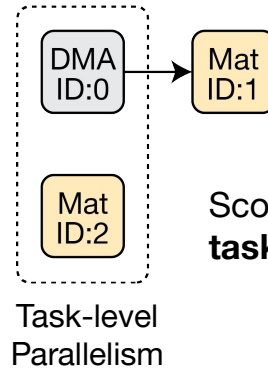
Mgmt Unit | Dataflow and Scheduling

Mgmt unit **②** dispatches dataflow graphs and **③** optimally schedules tasks



```
invoke.async DMA
invoke.async MATMUL, /*deps=*/dma
invoke.async MATMUL
```

Dependency Graph



Chain operations without SIMT having to come in between

Scoreboard extracts **parallel tasks** dynamically at runtime

Flexible **dataflow graph** of units

②

Optimal **scheduling** to maximize utilization

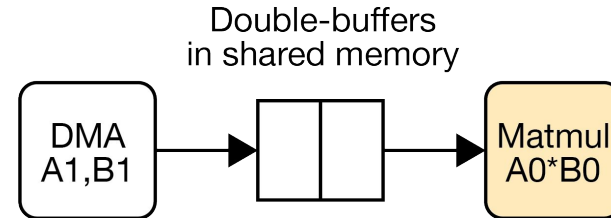
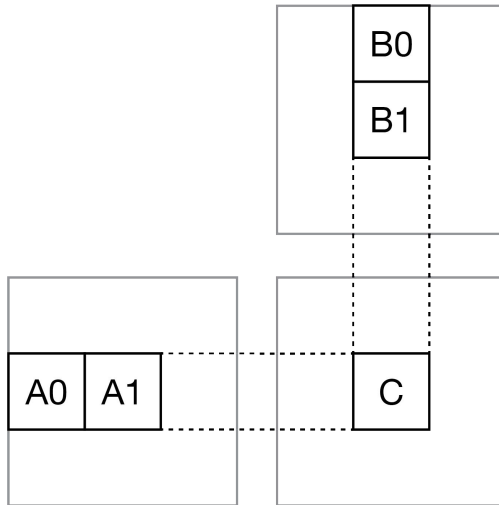
③



Use-case | GEMM with Pipelined DMA

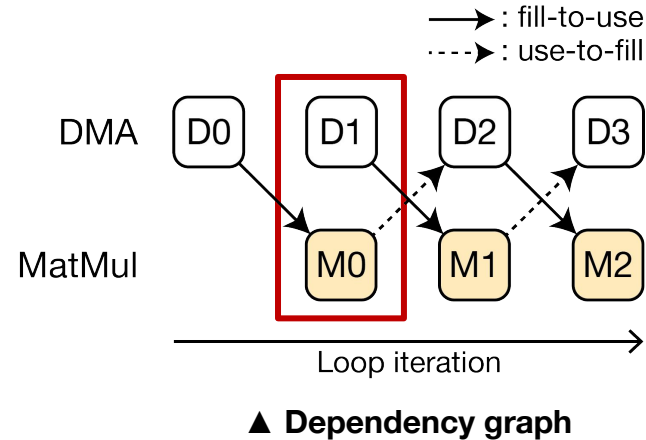
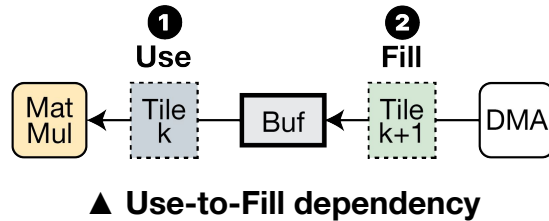
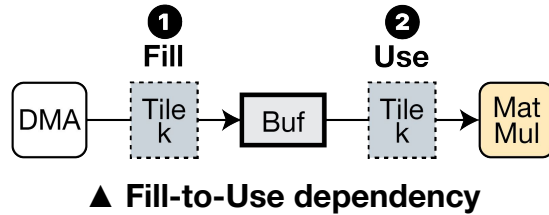
Output-tiled GEMM with each tile fully accumulating K dimension in a loop;
K-loop is **pipelined** between data movement and compute

Models **single-producer-single-consumer** with **pipelined decoupled access/execute**



Double-buffering has dual-dependency for buffer **fill-to-use** (DMA $\leftarrow$ Mat), and **use-to-fill** (Mat $\leftarrow$ DMA)

Scoreboard dispatches **DMA** and **matmul** in the same loop iteration **in parallel**, achieving **pipelined parallelism**

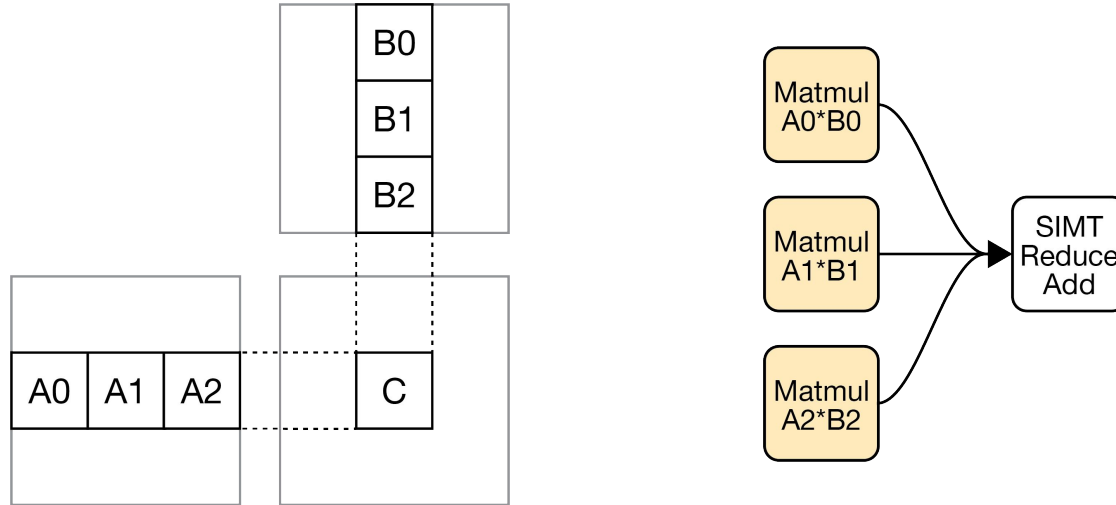




Use-case | GEMM with Split-K

Output-tiled GEMM with K dimension split into multiple parallel matmul operations;
Requires **reduce-add** operation of partial sums

Multi-producer-single-consumer dependency graph

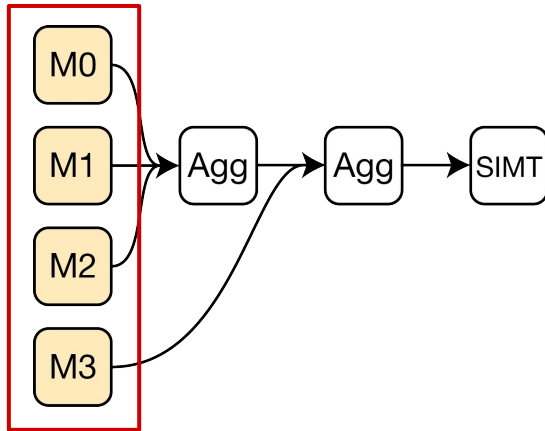




Use-case | GEMM with Split-K

Can specify arbitrary N producers to a consumer using a **tree**-based dependency graph
— `invoke.async` specifies max 3 dependent tasks

Scoreboard ensures producer tasks is **dispatched in parallel**,
and SIMT consumer task (reduce-add) is dispatched **only when all producers finish**



▲ Dependency graph

```
invoke.async EMPTY, /*id=*/agg0,  
                  /*deps=*/m0,m1,m2  
invoke.async EMPTY, /*id=*/agg1,  
                  /*deps=*/agg0,m3  
...  
// synchronization point  
invoke.sync SIMT, /*id=*/simt,  
                /*deps=*/agg1
```

▲ SIMT kernel instructions

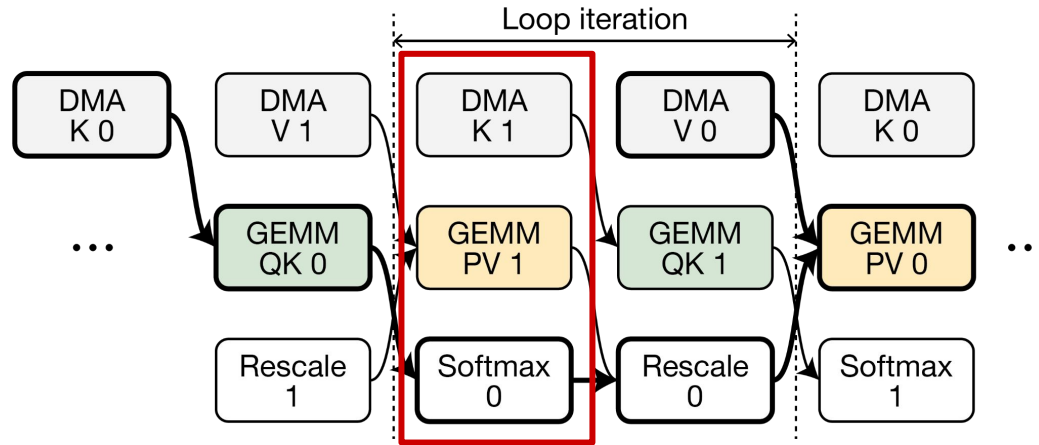


Use-case | Fused DNN Kernels

Fused DNN kernels feature more complex dependency graph due to more # of operations

Ex) **FlashAttention-3**: Pipeline of DMA → GEMM-QK → Softmax → GEMM-PV

SIMT threads participate in data-parallel computation, not just task invocation



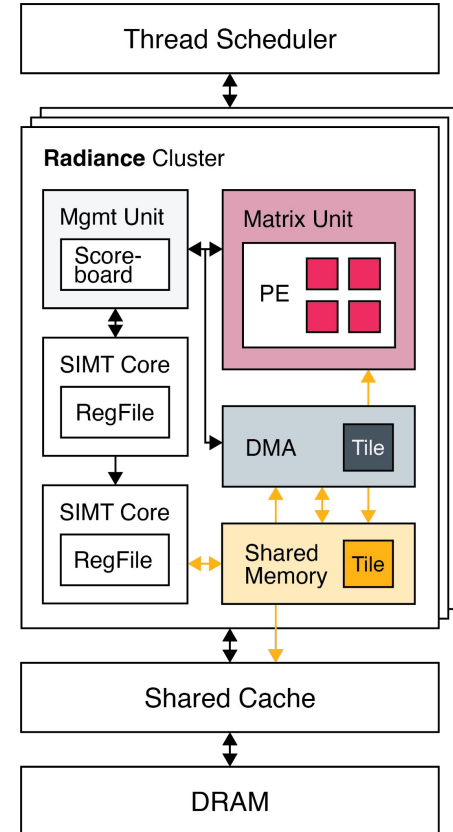
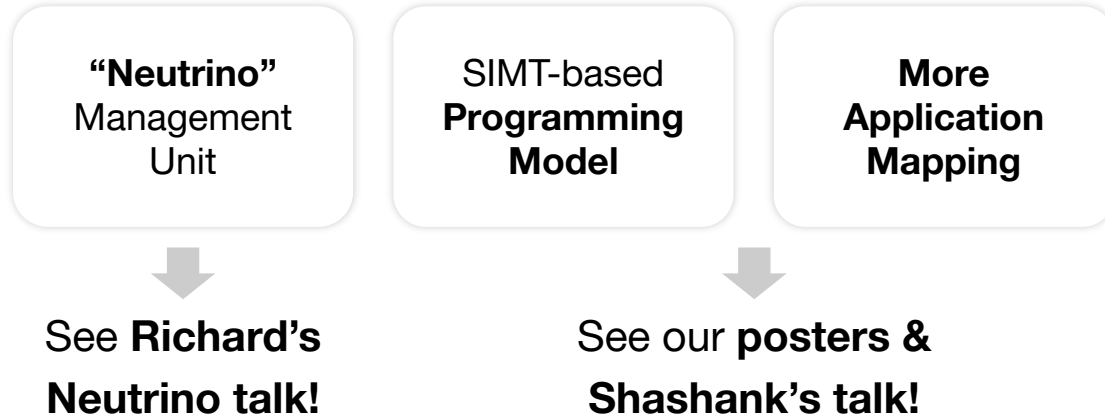
▲ Dependency graph



Conclusion

- Modern GPUs feature **asynchronous decoupled specialized units** to meet scalability demand
- How to **orchestrate** the decoupled units in GPUs in an efficient and programmable way?

Radiance, a new GPU design that consists of:





Thank You!